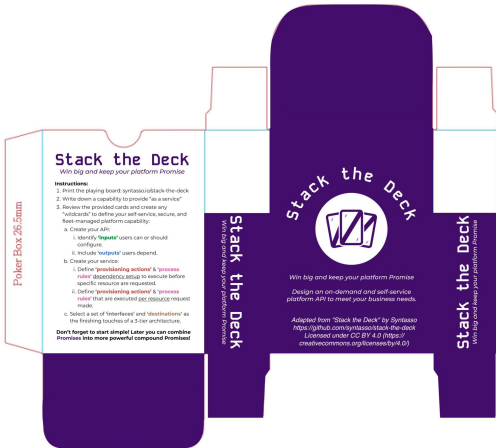






Stack the Deck

Win big and keep your platform Promise

背景

効果的な内部開発者プラットフォームは、事業拡大に伴い、業務のスピードアップ、安全性の向上、そして効率化を実現します。

効果的なプラットフォームを構築するには、以下の 3つの基本原則を取り入れる必要があります：

1. セルフサービスとオンデマンドアクセス
2. 一貫して適用されるビジネスプロセス
3. 大規模なサービス群の管理

内部開発者プラットフォームは、共通サービスを提供することで、これらを実現し、さらに多くの機能を提供します。このカードデッキを活用して、プラットフォームの基本原則をすべて満たす独自のサービスを設計してください。



Interface

インターフェースの説明

インターフェースにより、ユーザーはプラットフォームの機能を作成、更新、削除、保守、および確認することができます。

プラットフォームにアクセスする唯一の正しい方法というものは存在しません。スクリプトが必要なタスクもあれば、視覚化に依存するタスクもあり、それぞれ異なるインターフェースを必要とします。

重要なのは、最小限のメンテナンスで、プラットフォームがあらゆる種類のインターフェースに対応できるようにすることです。



Interface

API (デフォルト)

今日のあらゆる最新プラットフォームは、強力な APIを基盤とし、その上に追加のインターフェースを構築しています。例えば、クラウドプロバイダーはAPIを基盤とし、さまざまなユースケースに対応するためにCLIやGUIへと拡張しています。

したがって、このインターフェースは「デフォルト」と見なされ、常にインターフェースのセクションに含める必要があります。



ポータル

多くの場合、プラットフォームのサービスやリソースの一覧を表示するブラウザベース (GUI) のインターフェースです。

ポータルは、カスタム構築されたセルフホスト型プロジェクト (例: 人気のCNCFプロジェクト「Backstage」) や、SaaS製品 (例: Port、Cortex、Roadie) として提供されています。

ポータルの設定やデータはプラットフォームに直接依存するため、オーバーヘッドを削減するには、プラットフォーム自体によって管理されるべきです。



チケット管理システム

チケット管理システムといえば、長い待ち時間や手作業を連想しがちですが、組織のユーザー体験や追跡メカニズムの中核をなすことも少なくありません。

JiraやServiceNowなどの既存ツールを通じてプラットフォームを連携させることで、ユーザー体験や追跡メカニズムを維持しつつ、手作業をオンデマンド型のサービスに置き換えることが可能です。



Interface

コマンドラインインターフェース (CLI)

CLIを利用することで、ターミナルベースのプラットフォーム操作が可能になります。これはエンジニアにとって好まれる方法であるだけでなく、ブラウザにアクセスできない CI/CDスクリプトなどの一部のタスクにおいては必須条件となることもあります。

CLIの設定やデータはプラットフォームに直接依存するため、オーバーヘッドを軽減するには、プラットフォーム自体によって管理されるべきです。



統合開発環境 (IDE)

開発者は、グラフィカルな環境 (JetBrainsやVSCodeなど) であれ、ターミナル環境 (VIMやNeoなど) であれ、IDE上でAIやコードを扱うことがよくあります。

プラットフォームは、あらゆる IDE向けにMCPサーバーやプラグインを提供することで、より迅速なフィードバックや、ツールや開発環境への容易なアクセスを可能にします。



Destination

デプロイ先の説明

デプロイ先とは、宣言型コードのデプロイをスケジュールできる場所のことです。

プラットフォームには多数のデプロイ先が存在します。各デプロイ先は、GitOpsやその他のデプロイプロセスを通じて宣言型コードを適用できるターゲットロケーションを指します。

デプロイ先の種類は、技術の種類、環境、所有権モデルなどによって範囲を定義できるため、その数は無数にあります。



Kubernetes クラスタ

クラスタの数だけデプロイ先を設定できます。それぞれのデプロイ先は、そのユースケースに適したセキュリティレベルとスケーラビリティに合わせて構成されます。

各デプロイ先には、環境(開発／本番)、タイプ(GPU、PCI準拠)、または組織内でクラスタを分類するために用いるその他のドメインモデルを指定できます。



クラウドプロバイダー

プロバイダー、アカウント、さらにはリージョンごとに異なる環境をカバーするために、複数のクラウド先が必要になる場合があります。

各クラウド先は固有のドメインを表しており、独自のツール (CloudFormation や Bicep など) や、Crossplane や Terraform といった一般的な「Infrastructure as Code」ツールを使用して設定できます。



Destination

Software as a Service (SaaS)

これは、あなたの会社が依存しているソフトウェアを運用するベンダーであり、インターネット経由でアクセスする、あらゆるベンダーを表すことができます。

このデプロイ先を使用することで、利用可能なベンダー機能の管理や、特定のテナントへのアクセス権や機能の提供を行うことができます。



オンプレミス / ハードウェア

すべてのリソースがクラウド上にあるわけではありません。

オンプレミスのインフラストラクチャを管理する必要がある場合は、cronジョブのようなシンプルな手段を使って、デプロイ先で定義された状態を参照し、ネットワークデバイスからサーバー、メインフレームに至るまで、あらゆるリソースの設定を維持・更新することができます。



インプットの説明

インプット項目とは、プラットフォームのユーザーが設定できる情報のことです。

インプットフィールドには、列挙型、文字列、数値、ブール値など、さまざまな形式があります。

インプット項目はユーザーがプラットフォームとやり取りするための手段であるため、共同作業の一環として捉える必要があります。



名前

これは最も一般的なインプット項目であると同時に、多くの場合、最も不要なインプット項目でもあります。そのため、使用前によく検討してください。ユーザーから名前を入力してもらう必要がありますか？それとも、自分で生成できるものでしょうか？

名前を入力する項目は、プラットフォーム外でその名前が頻繁に参照される場合に最も重要になります。



サイズ

リソースの規模は、そのコストや効率性に影響を与える可能性があります。ユーザーは、バケットのサイズ(例:小、中、大)で要件を指定できる場合もあれば、具体的なバイト数を指定する必要がある場合もあります。

サイズを指定する場合、そのサイズが大きすぎたり小さすぎたりした場合は、実際のニーズに合わせてリクエストを更新する必要があるのはユーザー自身であることに留意してください。



物理的な場所

多くの場合、ユーザーはリソースがどこにデプロイされているかには関心がなく、ネットワーク経由で利用可能であればそれで十分です。

しかし、場合によっては、ユーザーがデータレジデンシの要件やレイテンシ要件を満たす必要があり、そのためリソースを特定の物理的な場所に配置しなければならないこともあります。



コンテナイメージ

ユーザーがプラットフォームに対してソフトウェアのデプロイを依頼する場合、そのユーザーはデプロイするソフトウェアを用意する必要があるでしょう。その方法の一つとして、レジストリやその他の手段を通じて、あらかじめ構築済みのイメージを提供することが挙げられます。



セキュリティレベル

すべてのリソースに同じレベルのセキュリティが必要というわけではありません。たとえば、顧客データに一切触れないリソースもあれば、顧客情報だけでなく、金銭的被害やその他の個人攻撃に悪用される可能性のあるデータを含むリソースもあります。

プラットフォームが必要なセキュリティレベルを自動的に判断できない場合、ユーザー自身がその情報を指定することができます。



依存関係

リソースには、1つ以上のオプションの依存関係が存在する場合があります。たとえば、ソフトウェアアプリケーションは、フィーチャーフラグ設定サービスやデータベースを利用するように設定できます。

ユーザーは依存関係のリストを指定することで、プラットフォームは既存の依存関係にアクセスしたり、要求されたリソースに関連して新しいインスタンスを作成・管理したりすることができます。



アップグレードルール

プラットフォームの利点は、全システムを一括してアップグレードできる点にあります。しかし、プラットフォーム全体を同時に更新できるからといって、必ずしもそうすべきとは限りません。

ユーザーによっては、アップグレードのペースを緩やかにしたり、更新時期を遅らせたりすることを希望する一方、最先端のリリースをすぐに導入したいと考えるユーザーもいるでしょう。



リポジトリ

ユーザーがプラットフォームに対してソフトウェアのデプロイを依頼する場合、そのユーザーはデプロイするソフトウェアを提供する必要があるでしょう。一つの方法として、ソフトウェアが格納されているリポジトリを用意し、プラットフォームがそのリポジトリからデプロイ情報を取得できるようにすることが挙げられます。



所有権

所有権は、タグ付けやアクセス権限の設定など、さまざまな用途に利用できます。

プラットフォームがリクエスト元のユーザーから所有権を特定できない場合、業務プロセスに準拠するために、所有権を明示的に指定する必要がある場合があります。



バージョン

リソースのバージョンに対する所有権は、アップグレードやメンテナンスの責任を暗黙的に決定づけます。

例えば、プラットフォームがバージョンを入力パラメータとして公開している場合、ユーザーはリソースのアップグレードに対応する必要があります。しかし、プラットフォームがメジャーバージョンのみを公開している場合、マイナーバージョンおよびパッチバージョンの更新管理は、暗黙的にプラットフォーム側が担うことになります。



設定情報

設定情報とは、定義された範囲内でリソースの動作に影響するあらゆるフィールドを総称して指すものです。

この入力項目を使用する場合は、具体的にどのような設定情報を公開するかを明記することを検討してください。



環境情報

ソフトウェアのビルド、テスト、デプロイを行う際、複数の異なるセットアップや構成が存在するのが一般的です。

環境によっては、同じインフラストラクチャやツールの構成要素が必要となる場合もありますが、その実装方法はしばしば異なります。こうした情報を基に、プラットフォームはユーザーに代わってこれらの構成を決定することができます。



アウトプットの説明

アウトプットとは、プラットフォームがリソースの作成時およびその後の経過に応じてユーザーに報告する情報のことです。

通常、人間が読み取ることを想定したデータと、機械が解析することを想定したデータの両方が含まれます。

これらの出力は、ユーザーがリソースを利用し、デバッグを行う上で十分なものである必要があります。



シークレット

リソースへのアクセスにシークレット(パスワードやデータベース接続文字列など)が必要な場合、ユーザーはそのシークレットの内容を知っておく必要があります。

APIのレスポンス自体は転送中に保護されていませんが、シークレットが利用可能な場所(例: HashiCorp Vaultやクラウドシークレットマネージャーの URLなど)を返すことは可能です。



オブザーバビリティの詳細

プラットフォームは、ユーザーとプラットフォーム提供者の間に共同責任を生み出します。ユーザーは自身のリソースを管理し、適切な構成(サイズやセキュリティレベルなど)をリクエストしていることを確認する必要があります。

これらの要件を管理するため、プラットフォームは必要なオブザーバビリティデータやツールへのアクセスを提供する場合があります。



アクセス情報

リソースは、多くの場合、ユーザーや他のシステムによって物理的に依存しており、アウトプット情報によって下流の依存関係の詳細が提供されることがあります。

ユーザーは、手動テストを実行するために URL やエンドポイントを知る必要がある場合があります。あるいは、別のリソースが、接続して上位レベルのシステムを構築するために、IP アドレスやその他の識別情報を知る必要がある場合もあります。



健全性チェック

「まだ着かないの？」という問いかけは、長距離ドライブに限った話ではありません。プラットフォームにおいて最も頻繁に聞かれる質問の一つは、リソースがいつ利用可能になるか、そしてそのライフサイクルを通じて常に利用可能かどうかということです。

健全性チェックの更新情報により、ユーザーはリソースの過剰または不足、セキュリティアラート、および問題発生時のデバッグ支援を含むその他の運用改善策について通知を受けることができます。



非推奨機能

サービスには継続的なサポートが必要です。アップデートにはユーザーの操作が必要な場合もあれば、プラットフォーム側で完全に処理される場合もありますが、ユーザーにはその旨を通知したい場合があります。

アウトプット情報として、廃止予定のバージョンについてユーザーに通知し、アップグレードの実施時期や方法について明確に伝えることができます。



Process rules

プロセスルールの説明

プロセスルールとは、リソースのプロビジョニングと並行して実行されるビジネスタスクのことです。

プロセスルールは、リクエストチケットを通じて手動でリクエストに対応する場合の「ToDoリスト」の一部と捉えることもできれば、デリバリーパイプラインに組み込むことも可能です。

これらは、依存関係の設定やリソースごとの設定（あるいはその両方）で利用できます。



Process rules

セキュリティスキャン

リソースは、多くの場合、導入前に静的スキャンを、使用中は定期的な動的スキャンを必要とします。

このスキャンは、ライフサイクルイベント時だけでなく定期的なスケジュールでも実行できるため、これらいずれかのスキャン、あるいは両方のスキャンとして利用できます。

また、このプラットフォームでは、スキャンの結果に基づいてさまざまなアクションを実行することも可能です。



Process rules

ステータスチェック

リソースの作成、更新、または削除の過程において、プラットフォームはそのリソースや他のリソースの状態を把握する必要がある場合があります。

例えば、プラットフォームには、優先度高のインシデントが発生している間はデプロイを行わない、あるいは一度に 1つのアップグレードのみを実行するといったルールが設定されている場合があります。



Process rules

利用可能オプションの取得

オプションは動的に変更される場合があります、プロビジョニング時に検証が必要になることがあります。

例えば、プラットフォームが一時的なテスト環境の負荷分散を行っている場合、プラットフォームは現在のテスト環境の状態を取得して、新しいアプリケーションをどこにデプロイすべきかを判断することができます。あるいは、プラットフォームが認証管理のために外部のLDAPサービスを利用する必要がある場合もあります。



Process rules

通知

リソースをリクエストしたユーザーだけでなく、その他の関係者に対しても、チャットボット、SMS、メール、その他の通知を送信することができます。

通知は、ユーザーへの情報提供や注意喚起に利用できます。リソースの監査が必要な場合は、レポートプロセスアクションの利用をご検討ください。



Process rules

予算チェック

組織は、ユーザー、チーム、または組織全体のコストを長期的に追跡する際に、FinOpsを実践することがあります。このプラットフォームは、FinOpsの要件への準拠状況を確認し、その結果に基づいてリソースに対して「許可」「拒否」、または「承認待ちのアラート」のいずれかの措置を講じることができます。



ステータス更新

プロビジョニングの終了時にステータスを更新するのが一般的ですが、進行状況を示すために、プロセス全体を通じてステータスを更新したい場合もあるでしょう。

ステータスは、単なる情報提供用である場合もあれば、必要なアクションを示す場合もあります。



Process rules

手動による承認

完全な自動化を目指しているとはいえ、コンプライアンス上の理由やこれまでの慣例から、手動による承認が依然として行われており、また行われるべき場合もあります。

このプロセスでは、権限を持つ担当者がリクエストを承認するまで一時停止し、待機することができます。



Process rules

リソースのタグ付け

リソースにタグやラベルを付けることで、所有者の特定などが容易になります。

これらのラベルは、運用部門がリソースを管理する上で役立つだけでなく、組織全体のリソースに対して監査を行う際、財務部門やセキュリティ部門、その他の部門にとっても有益です。



Process rules

スケジューリングルール

宣言型プロビジョニングを使用する場合、リソースを適切なデプロイ先にスケジューリングする必要があります。

デフォルトでは、リソースは利用可能なデプロイ先のいずれかにスケジューリングされますが、場合によっては、プラットフォームがより具体的なルールを適用し、スケジューリング先の特定のデプロイ先を制御することがあります。



Process rules

非推奨機能の管理

プラットフォームは、古くなった依存関係を特定し、その緊急度を判断し、影響を受ける関係者に通知し、安全なアップグレードを実施するためのプロセスを順守する必要があります。

プラットフォームは、プラットフォーム上のすべての新規および既存のリソースについて、コンプライアンス要件を順守する必要があります。



Process rules

レポート

コンプライアンス規制やその他の要件によりユーザーのリクエストを追跡する必要がある場合は、プラットフォーム内でのリクエストや変更に関する監査イベントを送信するように設定できます。さらに、プラットフォームはリソースの状態について定期的に自動レポートを作成することも可能です。

単にユーザーへの通知を目的とする場合は、別のオプションとして通知をご確認ください。



Provisioning action

プロビジョニングアクションの説明

プロビジョニングアクションにより、具体的で実用的なプラットフォームサービスが生成されます。

プロビジョニングには、命令型と宣言型があります。命令型は段階的な手順を実行します。宣言型は望ましい状態を定義します。宣言型プロビジョニングでは、デプロイ先が必須となります。

これらは、依存関係の設定でも、リソースごとの設定でも(あるいはその両方でも)使用できます。



Provisioning action

インフラストラクチャ

インフラストラクチャとは、コンピューティング、データストレージ、ネットワーク、その他のリソースを実行できるクラウドまたは物理ハードウェアのことです。

インフラストラクチャは、完全な単体（アップルパイ全体のようなもの）として提供される場合もあれば、共有リソースの一部（パイの一切れのようなもの）として提供される場合もあります。いずれの場合も、定義されたインフラストラクチャを割り当てることになります。例としては、サーバー、データベース、Kubernetesクラスターなどが挙げられます。



Provisioning action

Software as a Service (SaaS)

SaaS製品の設定を管理します。たとえば、ユーザーアカウントの追加、リソースの作成、設定の更新などです。



Provisioning action

Kubernetes クラスター

リソースはKubernetesにデプロイ可能ですが、追加のプロビジョニングを行うためにはKubernetesクラスターの作成が必要になる場合があります。



Provisioning action

認証情報

要求されたリソースを作成およびアクセスするために使用する認証情報を作成または取得します。

認証情報は、1つ以上のユースケースにおいて、1人以上のユーザーによって使用される場合があります。そのため、認証情報には複数の場所からアクセスしたり、複数のリクエストで参照されたりする必要がある場合があります。



Provisioning action

クラウドアカウント

リソースはクラウドプロバイダーにデプロイすることができ、追加のプロビジョニングを行うためにアカウントの作成が必要になる場合があります。



Provisioning action

Kubernetes 名前空間

リソースはKubernetesにデプロイ可能ですが、追加のプロビジョニングを行うためにはKubernetes名前空間を作成する必要がある場合があります。



Provisioning action

設定情報

設定情報とは、リソースに対する固有の設定を指します。

リソースには、グローバル設定やユーザーごとの設定が存在する場合があります。ユースケースによっては、他のリクエストの設定と組み合わせる必要がある場合があります。



Provisioning action

Git リポジトリ

プラットフォームが新規サービスの立ち上げ機能を提供している場合、初期のサンプルアプリケーションやデプロイメントも併せて提供するのが一般的です。

この処理は、単にリポジトリを作成してユーザーに引き渡すという命令型のアプローチで行うことも、プラットフォーム側が特定の設定を把握し、場合によっては制御することで継続的なサポートを提供する宣言型のアプローチで行うこともできます。



Provisioning action

IDとアクセス

IAM(別名: Authz)は、適切なユーザーが適切なリソースのみにアクセスできるようにするための、一連のルールと権限のセットです。

ルールによっては複数のユーザーに適用されるものもあれば、より具体的なものもあります。例えば、すべてのユーザーに 2要素認証を義務付ける一方で、特定のリソースへのアクセスはリクエストを行ったユーザーに限定されるといったケースが考えられます。



Provisioning action

既存のリソース

一部のリソースは、複数の依存関係とライフサイクルを共有しています。たとえば、アプリケーションは常にそのデータベースとともにプロビジョニングおよび廃止される場合があります。

ルールによっては、プラットフォームはアプリケーションに対する単一のリクエストを受け入れ、そのアプリケーションに関連するデータベースのみを残すことを選択する場合があります。